

ISFA2 — 2023-2024 — projet de simulation

Date limite : samedi 2 décembre 2023, 4h du matin

Approximation de la probabilité de débordement d'un fleuve.

1 Modèle

1.1 Contexte

On considère le fleuve *Coulantine* traversant la ville d'*Aussec*. On souhaite savoir quelle est la probabilité que ce fleuve déborde et inonde la ville au cours de l'année 2024, selon qu'on construise ou non des murets autour de son lit.

On modélise les pluies par un ensemble de dates où elles tombent (on considère ces pluies instantanées), et des intensités pour chacune de ces dates. À partir de ces données, aléatoires, un modèle nous permet de calculer la hauteur d'eau dans le fleuve à tout moment de l'année, et donc tester si la hauteur maximale dépasse un certain seuil correspondant au début d'une inondation.

1.2 Dates des pluies

On modélise les dates des pluies par un processus de Poisson inhomogène, de fonction intensité λ définie par

$$\lambda(t) = \lambda_0(1 + \alpha \sin(4\pi t)),$$

où λ_0 et α sont des constantes du modèle, vérifiant $\lambda_0 > 0$ et $0 \leq \alpha < 1$. Dans cette expression, t représente le temps en années, avec $t = 0$ au début de l'année 2024 et $t = 1$ à la fin de l'année 2024.

On note $T_1, T_2, \dots$ les variables aléatoires correspondant aux dates des pluies.

1.3 Intensité des pluies

On modélise les intensités de chacune des pluies par des variables aléatoires indépendantes, et indépendantes des dates des pluies, toutes de loi $\mathbb{P}_I$ de densité f_I définie par

$$f_I(x) = \mathbf{1}_{\mathbb{R}_+}(x) \frac{c_I}{m + |x - x_0|^\eta},$$

où c_I est une constante de normalisation, et $m > 0$, $x_0 > 0$ et $\eta > 2$ sont des paramètres du modèle.

On note $I_1, I_2, \dots$ les variables aléatoires correspondant aux intensités des pluies.

1.4 Hauteur d'eau

La hauteur d'eau $H(t)$ dans le fleuve *Coulantine* au temps t est définie comme suit :

$$H(t) = \sum_{i \geq 1} I_i \times r(t - T_i),$$

où r est la fonction de résorption de l'eau de ce fleuve :

$$r(t) = \mathbf{1}_{\mathbb{R}_+}(t) \exp(-vt)$$

en notant v la vitesse de résorption.

On notera $H_{\max}$ la hauteur maximale de l'eau dans le fleuve Coulantine au cours de l'année 2024 :

$$H_{\max} = \max_{0 \leq t \leq 1} H(t).$$

2 Résultat attendu

On cherche à approximer, par une méthode de Monte Carlo, la probabilité que $H_{\max}$ dépasse le seuil h_0 (donné) correspondant à l'inondation de la ville d'Aussec :

$$p^*(h_0) = \mathbb{P}[H_{\max} > h_0].$$

Les simulations devront être effectuées avec le logiciel R, en faisant, dans un deuxième temps, appel à une partie de code sous C/C++ (cette technique sera expliquée dans un autre cours).

Vous choisirez, pour vos essais, des valeurs de paramètres qui vous semblent raisonnables, et qui mènent à une valeur de p^* de l'ordre de 0.1.

2.1 Concours d'efficacité

Pour ce concours, chaque groupe devra proposer un code R pur (sans utiliser de code C/C++), qui effectue de la manière la plus efficace possible les trois opérations suivantes :

- Simulation de n valeurs suivant la loi $\mathbb{P}_I$.
- Approximation de $p^*(h_0)$.
- Approximation de $p^*(h'_0)$, où h'_0 est un seuil très grand, correspondant à la construction d'un muret autour du lit du fleuve.

Le programme R ne devra utiliser aucune bibliothèque spécifique (ne pas utiliser `library`).

Afin de comparer votre code à celui des autres groupes, vous pourrez déposer un fichier R sur <https://clarolineconnect.univ-lyon1.fr/> (dans Simulation/Projet/-Soumission). Les codes déposés seront régulièrement évalués et les scores obtenus seront publiés. Plusieurs membres du groupe peuvent déposer un code, mais un seul code sera évalué chaque jour pour chaque étudiant.

Le code présent sur la plate-forme à la date limite sera celui qui sera retenu pour le classement final des groupes (dans le cas de plusieurs codes présents, c'est le meilleur qui sera pris en compte), et l'attribution d'une note. En plus de la mesure de son efficacité, la validité du code sera bien entendu vérifiée.

Afin d'être évalué, le code déposé doit suivre les consignes suivantes :

2.1.1 Nom

Le fichier déposé doit s'appeler `GXX.R`, où `XX` est remplacé par le numéro de groupe. Par exemple, un étudiant du groupe 2 doit déposer un fichier nommé `G02.R`.

2.1.2 Syntaxe

Le fichier doit être syntaxiquement correct : on doit pouvoir appeler la commande `Rsource("GXX.R")` sans erreur.

Le fichier doit être encodé en UTF8 (en cas de doute, n'utilisez aucun caractère accentué ou spécial dans votre code, même dans les commentaires).

Par ailleurs, le code ne doit faire aucun calcul long, ne doit produire aucun graphique : il doit uniquement définir les fonctions demandées (et éventuellement des fonctions intermédiaires).

2.1.3 Simulation de la loi $\mathbb{P}_I$

Le fichier doit déclarer une fonction **rintensite** qui simule la loi $\mathbb{P}_I$, prenant les paramètres suivants :

- le nombre **n** de valeurs à simuler ;
- une liste **params** contenant les paramètres intervenant dans le modèle :

```
params$m = m
params$x0 = x0
params$eta = η
```

Cette fonction doit renvoyer un vecteur de taille **n** contenant les valeurs simulées suivant la loi $\mathbb{P}_I$.

Lors de l'appel de votre fonction, la valeur de **n** utilisée sera ajustée pour obtenir un temps d'exécution proche de 10 s (ce n'est pas à vous de faire cet ajustement).

2.1.4 Approximation de la probabilité $p^*(h_0)$

Le fichier doit déclarer une fonction **petoile**, qui calcule une approximation de $p^*(h_0)$ par une méthode de Monte Carlo. Elle doit prendre en compte les paramètres suivants :

- **n.simul** est le nombre de simulations à effectuer ;
- une liste **params** contenant les paramètres intervenant dans le modèle :

```
params$m = m
params$x0 = x0
params$eta = η
params$lambda0 = λ0
params$alpha = α
params$v = v
params$h0 = h0
```

La fonction **petoile** doit renvoyer une liste contenant l'approximation de la probabilité demandée, ainsi qu'un intervalle de confiance à 95 % associé à cette approximation, sous la forme

```
list(p=...,demi.largeur=...)
```

La valeur de **n.simul** sera ajustée pour obtenir un temps d'exécution proche de 60 s (ce n'est pas à vous de faire cet ajustement).

2.1.5 Approximation de la probabilité $p^*(h'_0)$

Le fichier doit déclarer une fonction `petoile.petite` similaire à `petoile` mais optimisée pour être plus rapide dans les cas où la probabilité à estimer est très petite. Cette fonction sera donc appelée avec une grande valeur de h'_0 .

2.1.6 Utilisation d'un code C à partir de R

Le fichier R doit également déclarer une fonction `petoile.c`, qui fonctionne de la même façon que `petoile`, mais qui fait appel à du code C, que ce soit à partir d'un fichier C à part ou en utilisant `Rcpp`.

- Si vous souhaitez utiliser un code C à part, joignez-le également à votre copie lors de votre soumission, en même temps que votre code R. Un code C devra être écrit dans un fichier ayant l'extension `c`, et un code en C++ devra être écrit dans un fichier ayant l'extension `cc` ou `cpp`. Pour l'évaluation de votre code, la machine à évaluation va appeler la commande `R CMD SHLIB` sur votre fichier C, puis appeler votre code R.
- Si vous souhaitez utiliser `Rcpp`, vous pouvez utiliser cette bibliothèque dans votre code R.

Le but est d'améliorer la vitesse de simulation, du mieux que vous pouvez, sans forcément reprogrammer tout en C.

2.2 Influence des paramètres

Étudier l'influence des paramètres λ_0 , α , x_0 , v et h_0 sur la probabilité $p^*(h_0)$. Le sens de cette influence était-il attendu ?

2.3 Rapport écrit et soutenance

Chaque groupe devra déposer, avant la date limite, un compte-rendu au format PDF exclusivement, sous le nom `GXX.pdf` (toujours en remplaçant `XX` par le numéro du groupe). Le compte-rendu devra aborder les éléments suivants :

- Organisation du travail en groupe : quel a été le rôle de chaque étudiant, comment vous êtes-vous coordonnés ?
- Quelles sont les méthodes de simulation que vous avez implémentées ?
- Quelles sont les techniques de programmation qui vous semblent intéressantes dans votre code ?
- Quelle est la partie du code que vous avez codée en C/C++, et pourquoi ? Cela vous a-t-il fait gagner en efficacité ?
- Comment avez-vous vérifié la validité de votre code (tests avec certaines valeurs de paramètres, calculs intermédiaires, représentations graphiques, etc.) ?
- Détailler les résultats de votre analyse de l'influence des paramètres décrite en 2.2.
- Quelles études basées sur ce modèle vous paraîtraient pertinentes ? Quelles évolutions de ce modèle proposeriez-vous, afin de le rendre plus réaliste ?